Digital Signal Processing

Assignment 2: FIR Filters

ECG Filtering, Adaptive Noise Cancellation and Heartbeat Detection

Patrick Connolly (2777107C)
Zabiallah Azar (2893681A)

Recordings and Code Structure

We each recorded an ECG in the lab. Patrick's recording (`ecg1.tsv`) is the signal filtered in every task, and Zabi's (`ecg2.tsv`) provides the matched filter template in Task 5. Column 8 of each file (counting from 0) holds the ECG in volts, which we convert to millivolts. The sampling rate comes from the time column.

Table 1: Recording parameters.

	`ecg1.tsv` (Patrick)	`ecg2.tsv` (Zabi)
Sampling rate f_s	250 Hz	250 Hz
Samples N	13 414	6985
Duration	53.7 s	27.9 s
Bin spacing f_s/N	0.019 Hz	0.036 Hz
Mean (DC offset)	−7.05 mV	18.81 mV
Mains peak	57.5 µV at 50.13 Hz	117.7 µV at 50.11 Hz

As asked on Moodle, design and realtime operation are kept apart. `fir_design.py` calculates the coefficients and `fir_filter.py` holds the realtime `FIRfilter` class. `ecg_plot.py` loads the recordings and does Task 1, and `fir_realtime_sim.py`, `lms_filter.py` and `heartbeat.py` are the main programs for Tasks 2–3, 4 and 5. The only signal processing calls we use from NumPy are `fft`, `ifft` and `hamming`, and every bin frequency is built by hand. All filtering goes through `FIRfilter` one sample at a time.

Task 1: ECG in the Time and Frequency Domain

Time domain. The R peaks are visible in both recordings (figs. 1 and 2), but they sit on a large offset that drifts. Patrick's averages −4.57 mV over the first second and −9.15 mV over the last. A 50 Hz ripple runs through both zoomed heartbeats and partly hides the P wave.

ECG. Most of the ECG lies between 0.5 Hz and 40 Hz (green). The heart rate sets the lowest component. Zabi's shows a clear fundamental at 1.2 Hz, and Patrick's is smeared over 0.97 Hz to 1.24 Hz because Patrick's rate varies between 58 and 75 bpm (Task 5). The largest component from 5 Hz to 40 Hz is 16.3 µV, falling to 1.7 µV from 40 Hz to 48 Hz.

Noise. Table 2 lists what we found in `ecg1.tsv` and what we did about it.

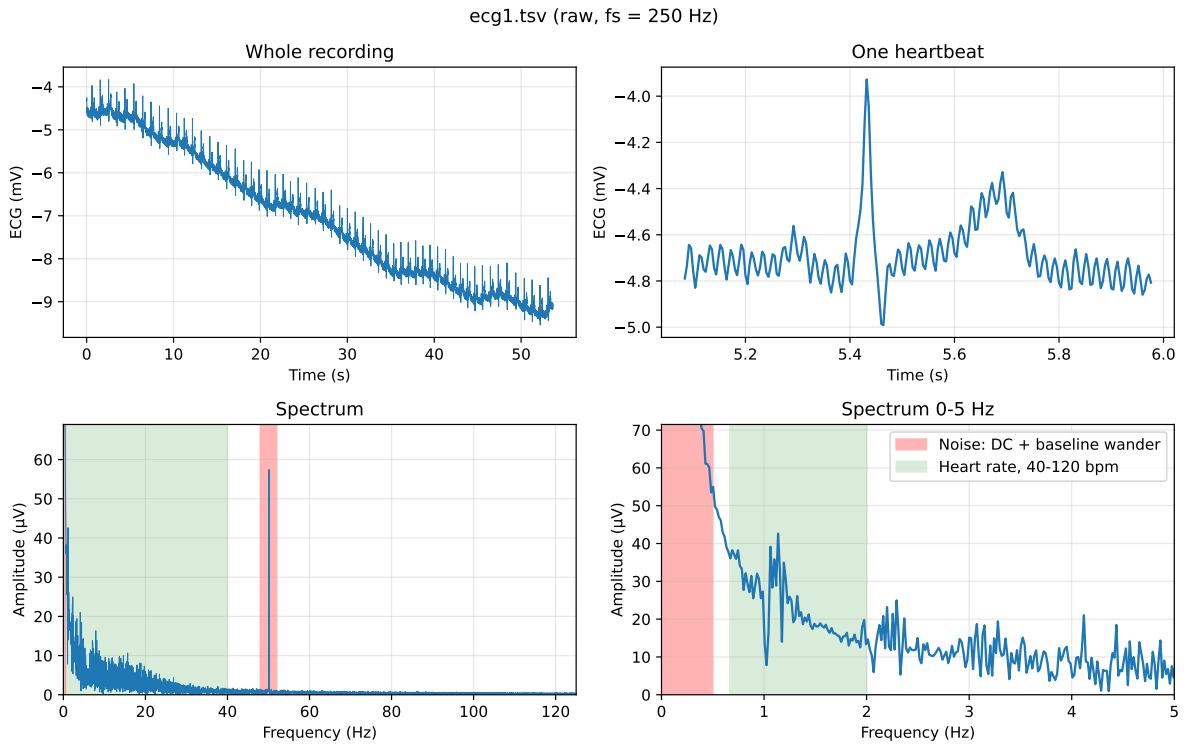


Figure 1: Patrick's raw ECG (`ecg1.tsv`). Linear spectra: the bands kept as ECG are shaded green, those removed as noise are red.

Table 2: Noise identified in the spectrum of `ecg1.tsv`.

Component	Evidence	Action
DC and baseline wander, 0 Hz to 0.5 Hz	up to 1.75 mV below 0.5 Hz, plus the DC offset	remove
Mains, 48 Hz to 52 Hz	sharp line, 57.5 µV at 50.13 Hz	remove
Mains harmonic, 100 Hz	no peak (see below)	leave
Above 52 Hz	flat floor, at most 1.3 µV	leave

Mains harmonics lie at multiples of 50 Hz, but at $f_s = 250$ Hz anything above the 125 Hz Nyquist frequency folds back: 150 Hz and 350 Hz land on 100 Hz, 200 Hz and 300 Hz land on 50 Hz, and 250 Hz lands on DC. Every harmonic can therefore only appear at 0, 50 or 100 Hz, and the first two are removed anyway. In `ecg1.tsv` there is no peak at 100 Hz: the largest value from 95 Hz to 105 Hz is 0.71 µV, at 95.9 Hz, so we do not add a 100 Hz stop band. Zabi's recording has a faint 1.2 µV line at 100.3 Hz, 40 dB below its mains peak, which is too small to affect the template. We do not low-pass at 40 Hz either, since the floor above 52 Hz is negligible and a low-pass would only risk rounding the QRS complex.

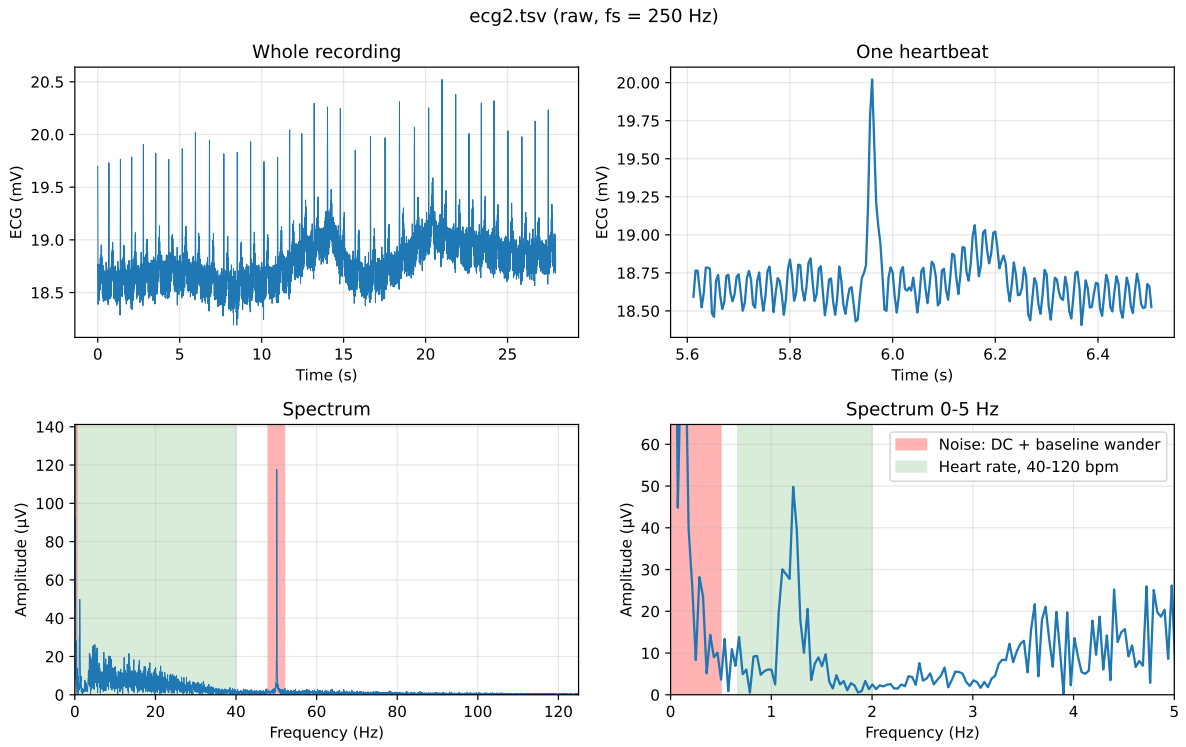


Figure 2: Zabi's raw ECG (`ecg2.tsv`), shaded in the same way as fig. 1.

Task 2: FIR Filter Design

`design_fir(fs, stop_bands)` takes the sampling rate and a list of (low, high) frequency ranges to remove. It follows the design steps in the lecture notes (Sec. 6.7.8). The desired response is filled in by hand on M frequency bins, with the upper half mirrored onto the lower half:

$$f_k = \min(k,\, M-k)\,\frac{f_s}{M}, \qquad H[k] = \begin{cases} 0 & \text{if } f_k \text{ lies in any stop band,} \\ 1 & \text{otherwise,} \end{cases} \qquad k = 0, \ldots, M-1. \quad (1)$$

The IFFT of H gives the impulse response with negative time in its upper half. Swapping the halves makes it causal, and a Hamming window then smooths the truncation to M samples:

$$h[n] = \mathrm{IFFT}\{H\}\Big[\Big(n - \tfrac{M-1}{2}\Big) \bmod M\Big]\; w[n], \qquad w[n] = 0.54 - 0.46\cos\frac{2\pi n}{M-1}. \quad (2)$$

Number of coefficients. The IFFT samples the response every f_s/M Hz, so a stop band of width Δf can only be represented if $f_s/M \le \Delta f$. The narrowest band sets the requirement, giving a minimum of $M = f_s/\Delta f_{\min}$, or 501 taps for our 0.5 Hz baseline band. This is only the best case, because the window smears each band edge over a few bins. With 501 taps the 0.5 Hz edge took 2.3 dB off the heart rate fundamental at 1 Hz, so we use twice the minimum, rounded up to the next odd number:

$$M = \text{smallest odd integer} \ge \frac{2f_s}{\Delta f_{\min}} = \frac{2 \times 250}{0.5} \;\Rightarrow\; M = 1001. \quad (3)$$

The bin spacing is then 0.25 Hz and the gain at 1 Hz is 0 dB. M is odd because a symmetric, odd-length filter has a linear phase and a delay of a whole number of samples, $(M-1)/2$, while a symmetric even-length filter is forced to zero at Nyquist, which a bandstop cannot have. The cost is a delay of 500 samples, or 2.00 s. This is set entirely by the baseline band: the 4 Hz wide mains band alone would need only 125 taps.

Edge cases. A band starting at 0 Hz includes bin 0, because the test in eq. (1) is $f_k \geq f_{\text{low}}$, so DC removal needs no special case. Windowing lets a little DC back in, but the gain at DC is only −82.3 dB, which takes the −7.05 mV offset down to a filtered mean of 0.6 µV. An earlier version subtracted the mean of h to force the DC gain to exactly zero, but that made no practical difference, so we removed it. Mirroring the bins in eq. (1) makes h real and symmetric. Basing M on the narrowest band means every band spans at least two bins, so none can fall between them. Any number of bands can be passed in: adding $(98, 102)$, for example, gives −51.6 dB at 100 Hz with no change to the code.

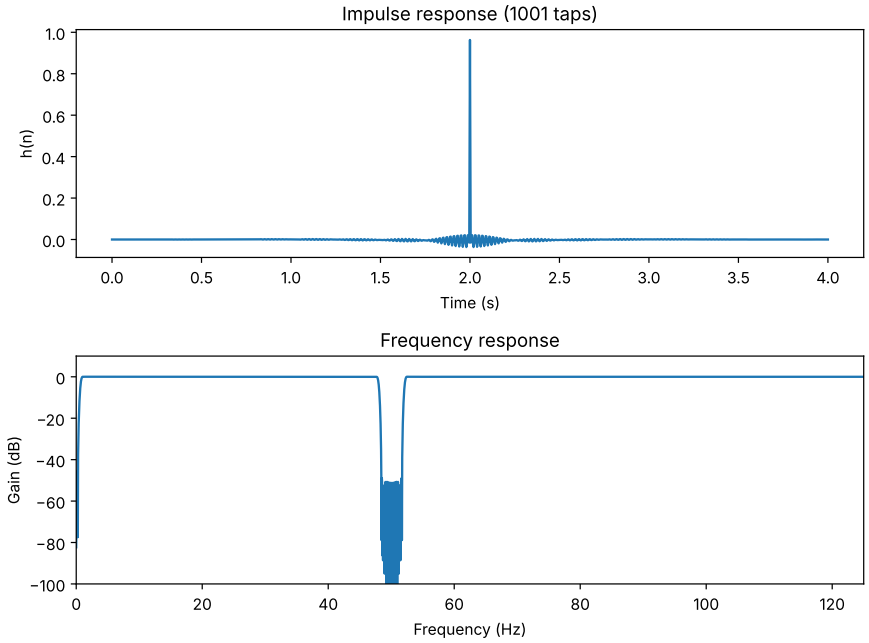


Figure 3: Designed filter: impulse response (top) and frequency response (bottom).

Table 3: Measured gain of the designed filter (fig. 3). Between 1.2 Hz and 47 Hz, and from 53 Hz up to Nyquist, the gain stays within ±0.03 dB.

f (Hz)	0	0.25	0.5	0.72	1.0	48	50.13	52
Gain (dB)	−82.3	−74.1	−12.7	−3.0	0.0	−3.4	−53.5	−9.5

Task 3: Realtime FIR Filter

`FIRfilter.dofilter(v)` takes one sample and returns one sample, evaluating

$$y[n] = \sum_{m=0}^{M-1} h[m]\, x[n-m] = \sum_{m=0}^{p} h[m]\, b[p-m] \; + \sum_{m=p+1}^{M-1} h[m]\, b[M+p-m], \tag{4}$$

where b is a ring buffer of length M and p is where the newest sample was written. Each new sample overwrites the oldest and the offset moves on, so one value is written per time step and nothing is shifted. Newest first, the delay line is $b[p], \ldots, b[0], b[M-1], \ldots, b[p+1]$, which is two continuous slices. `halves` splits h at the same point, so the filter is two dot products. That is 1001 multiplications per sample, or about 250 000 per second at 250 Hz, which is easily realtime.

We feed the ECG in one sample at a time. The first M samples (4 s) are left out of the plots while the delay line fills.

The ECG is intact. The drift of 4.6 mV is gone and the trace now stays between −0.33 mV and 0.73 mV (fig. 4). All five waves are visible in fig. 5. Relative to the R peak (0.68 mV), the P wave is at −140 ms, Q at −60 ms, S at +28 ms (−0.26 mV) and T at +256 ms (0.32 mV). Because the filter has linear phase, every frequency is delayed by the same 500 samples, so the shape of the beat does not change. To check this we measured each R wave's height above the level just before it. Averaged over all 54 beats, the filtered height is 1.2 % larger than the raw height, so the QRS amplitude is preserved.

Spectrum. The mains line drops from 42.9 µV to 0.09 µV over the plotted samples, which is −53 dB and agrees with table 3. Above 53 Hz the filtered curve sits about 10 dB below the raw one even though the filter's gain there is 0 dB. This is not filtering. The raw floor is spectral leakage from the drift, since the spectrum treats the record as periodic and the drift becomes a step at its ends. Removing a straight line from the raw signal takes its 95 Hz to 105 Hz median from −68.3 dB to −78.4 dB, which matches the filtered −78.6 dB.

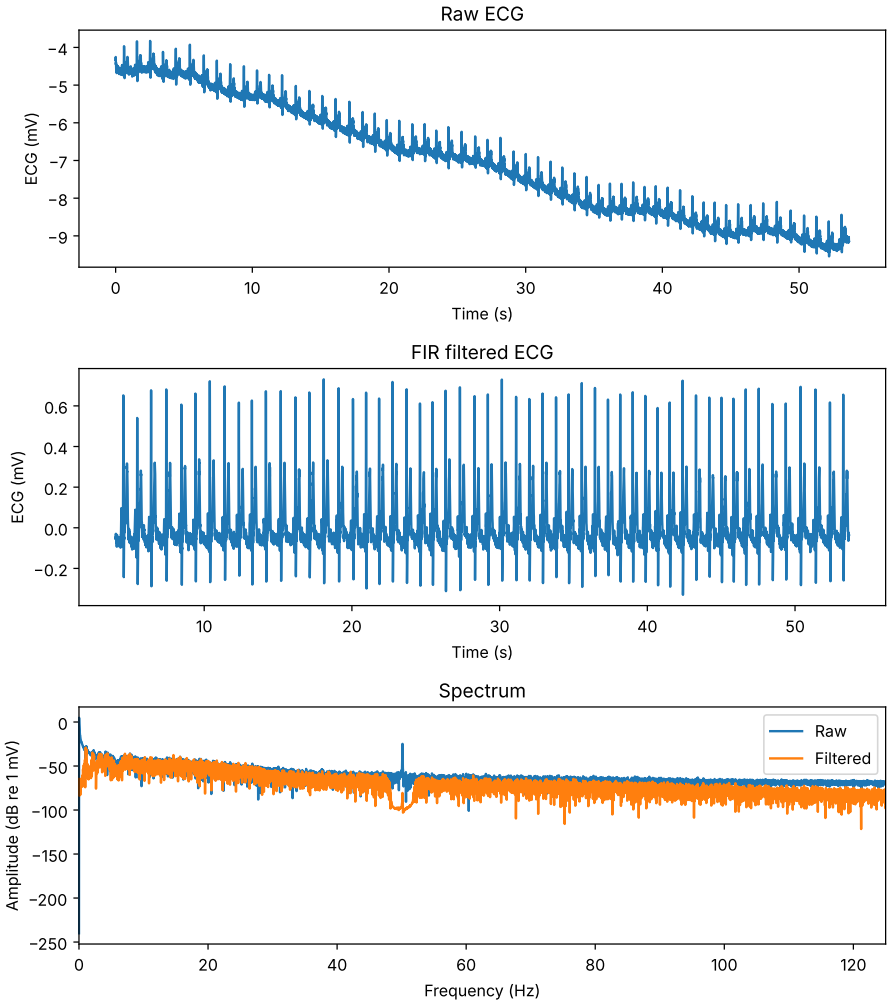


Figure 4: Patrick's ECG before and after the FIR filter, with the spectra of both. The spectra use a dB axis so that the notch is visible.

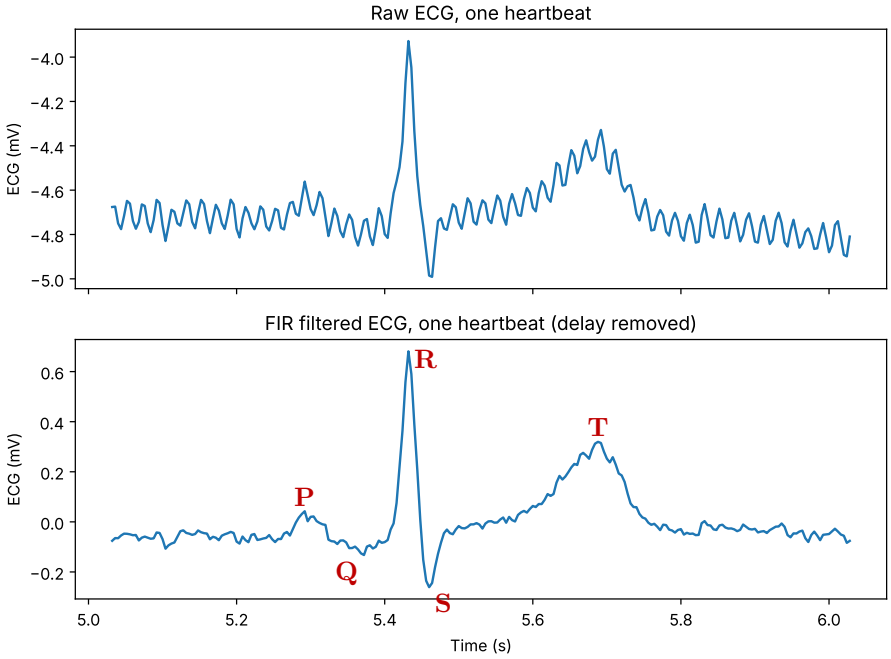


Figure 5: One heartbeat before and after filtering, with the filtered trace moved back by the 2.00 s delay so the two line up. The PQRST waves are labelled on the filtered trace.

Task 4: Adaptive LMS Filter

`doFilterAdaptive(signal, noise, learningRate)` also works one sample at a time. It stores the noise reference in the delay line, subtracts the filter output from the signal and updates each weight with the LMS rule (lecture notes, Eq. 147):

$$e[n] = d[n] - \sum_{m=0}^{M-1} h_m[n]\, x[n-m], \qquad h_m[n+1] = h_m[n] + \mu\, e[n]\, x[n-m]. \tag{5}$$

The filter can only cancel what is correlated with the reference, so the error $e[n]$ is the cleaned ECG.

The reference is a sine at the measured mains frequency, $x[n] = \sin(2\pi \cdot 50.13\, n/f_s)$. It carries no information about baseline wander, so $d[n]$ is the ECG after a highpass FIR designed with the baseline band only. That filter also has 1001 taps, so the LMS and bandstop outputs share the same delay and can be overlaid directly. The LMS has $f_s/50 = 5$ taps, one mains cycle, which is enough to set the gain and phase of the sine. We chose $\mu = 0.04$, so that $\mu MP = 0.1$ for a reference power of $P = \frac{1}{2}$. In our tests the filter became unstable at about $\mu MP = 2$. Raising μ below that limit removes more of the mains but distorts the ECG: at $\mu = 0.2$ the largest R peak grows from 0.77 mV to 0.88 mV, most likely because the weights start to react to the QRS complex itself.

The two outputs are almost identical in the time domain (fig. 6), with a correlation of 0.9992 and an RMS difference of 8.3 µV. Table 4 shows the differences are small. The bandstop goes slightly deeper at the mains frequency, and in the spectrum its notch is wider because it removes everything from 48 Hz to 52 Hz, whereas the LMS only removes what correlates with the sine. The LMS output has a slightly larger R peak, for the reason above.

The LMS saves nothing here, since baseline wander still needs the same 1001-tap highpass and therefore the same delay. Its advantage is that it does not need a fixed band. Even with a reference of exactly 50.00 Hz, 0.13 Hz away from the true mains frequency, the moving weights

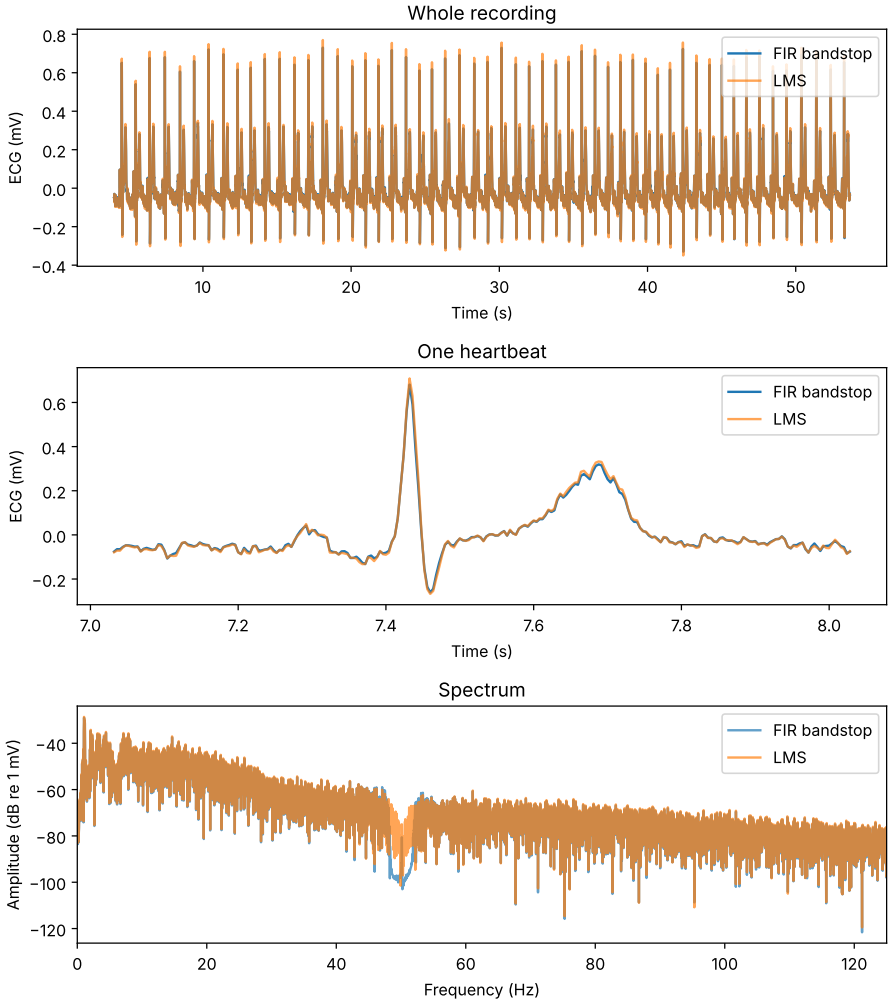


Figure 6: FIR bandstop and LMS outputs compared over the whole recording, for one heartbeat, and in the frequency domain.

Table 4: FIR bandstop and LMS compared. The mains residual is measured against the 43.7 µV left after highpass filtering alone.

	FIR bandstop	LMS
Filters	1001-tap bandstop	1001-tap highpass + 5-tap LMS
Delay	2.00 s	2.00 s
Multiplications per sample	1001	1011
Mains residual at 50.13 Hz	0.09 µV (−53.4 dB)	0.18 µV (−47.8 dB)
Largest value, 48 Hz to 52 Hz	0.40 µV	0.62 µV
Largest R peak	0.73 mV	0.77 mV

hold the residual to 3.0 µV, which is 23 dB below the original line. For this recording, where the mains is steady, we would keep the fixed FIR.

Task 5: Heartbeat Detection

The template r is one PQRST of $L = 175$ samples (0.3 s before the R peak to 0.4 s after) taken from Zabi's FIR-filtered ECG, with its mean removed. The matched filter is an `FIRfilter` whose coefficients are the time-reversed template (lecture notes, Sec. 6.7.10), and its output is squared so the peaks stand out and the sign does not matter:

$$h_{\mathrm{mf}}[n] = r[L-1-n], \qquad m[n] = \Big(\sum_{k=0}^{L-1} h_{\mathrm{mf}}[k]\, y[n-k]\Big)^2, \qquad \theta = 0.15\Big(\sum_{k} r[k]^2\Big)^2. \tag{6}$$

A beat is an upward crossing of the threshold θ. Our first version used 0.3 of the largest output in the whole recording, which a realtime system cannot know. θ now comes from the template alone, so it is fixed during setup, and it found the same 54 beats. The self-match in eq. (6) is what the template would give on an identical beat, and the factor of 0.15 allows for Patrick's beats being smaller than Zabi's (R peaks of 0.7 mV against 1.2 mV). The bandstop, matched filter, squaring and threshold all run inside one sample-by-sample loop.

The momentary heart rate at beat i is $60 f_s/(b_i - b_{i-1})$, where b_i is the sample number of the beat. As a heuristic, a crossing less than $60/200\, f_s = 75$ samples after the last beat would mean a rate above 200 bpm, so it is treated as noise. The detection itself is dropped rather than just the rate, so that the interval to the next real beat stays correct.

The detector finds all 54 beats, and the heart rate over the 53 intervals runs from 58.4 to 74.6 bpm with a mean of 65.5 bpm (fig. 7). The heuristic rejected nothing on this recording, because the margin is wide: the smallest beat peak is 10.7, the threshold is 5.3 and the largest output between beats is 1.2. Using a template from another person works well, with Patrick's beats reaching 30 % to 45 % of the self-match.

To check that the heuristic does work, we lowered the factor from 0.15 to 0.03 so that noise crosses the threshold. That produced 58 crossings, of which 4 were rejected, leaving 54 beats and the same heart rate range. At 0.02 there are 128 crossings and the heuristic is no longer enough, with rates of up to 188 bpm getting through. It is a safety net and does not replace a sensible threshold.

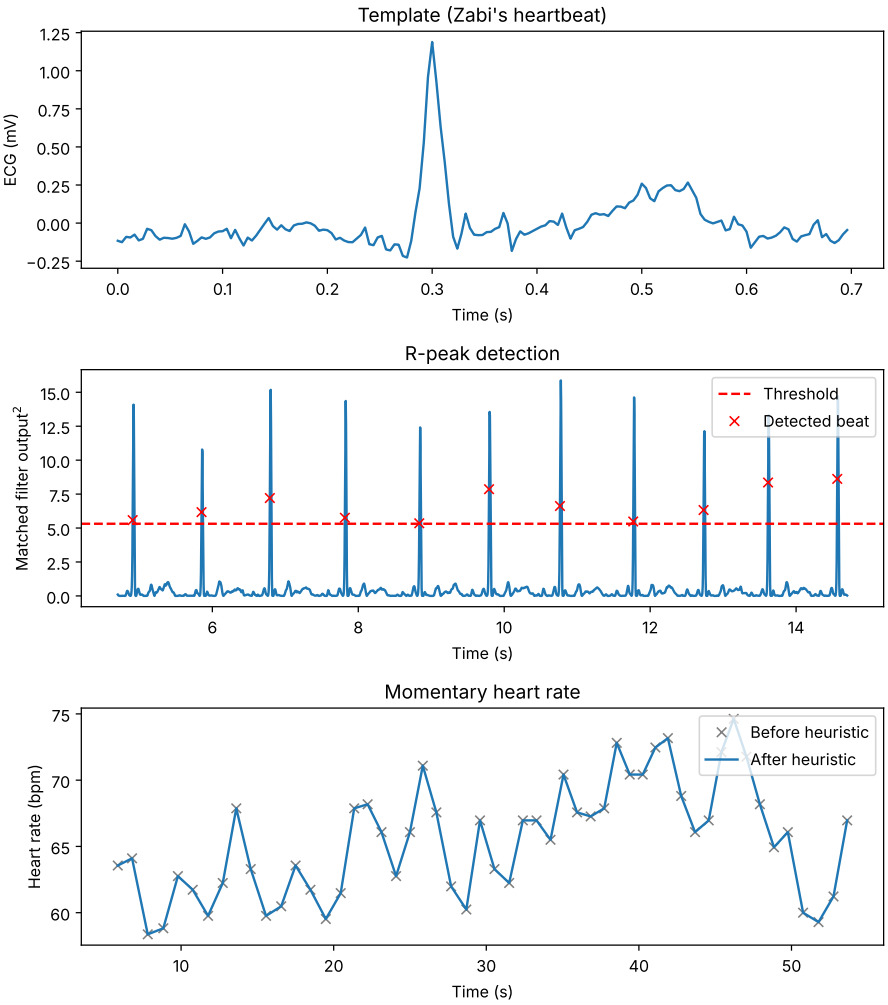


Figure 7: The template from Zabi's ECG (top), the squared matched filter output on Patrick's ECG for the first 10 s with threshold and detections (middle), and the momentary heart rate (bottom). No detection was rejected, so the two heart rate curves coincide.

Conclusion

The IFFT design with a Hamming window removes DC, baseline wander and mains while leaving the PQRST waves intact, with R wave height preserved to within about 1 %. The narrow 0.5 Hz baseline band sets the length of the filter, and the price of keeping the heart rate fundamental untouched is 1001 taps and a 2 s delay. The ring buffer keeps the realtime cost to one write and M multiplications per sample. The LMS filter removes the mains almost as well as the fixed bandstop and can cope with a reference that is slightly off, but it still needs the FIR highpass and so has no advantage in delay here. The matched filter with a template from the other partner and a threshold fixed in advance detects all 54 beats, with no false detections for the heuristic to remove.

Generative AI Acknowledgement

Claude Code (model Claude Opus 5.5, Anthropic, November 2026) was used to draft the text and LaTeX of this report from our code, figures, commit history and recordings, and to measure the numbers quoted from them. **[TO COMPLETE: say whether GenAI was used for the Python code (tool name, version, publisher, and how it was used), and what you did**

yourselves, e.g. recording the ECGs, the design decisions, writing and optimising the code, and checking this report.]

Appendix: Python Code

ecg_plot.py

```python
import os
import numpy as np
import matplotlib.pyplot as plt

HERE = os.path.dirname(os.path.abspath(__file__))  #file finder
MINE = "ecg1.tsv"
ZABI = "ecg2.tsv"  # Zabi recording for matched filter template
ECG_COLUMN = 8  #ecg voltage

# noise picked out forFIR
# no 100 Hz band: at fs = 250 Hz all mains harmonics alias onto 0, 50 or 100 Hz and there is no peak at 100 Hz
BASELINE = (0, 0.5)
MAINS = (48, 52)
MAINS_HZ = 50.13  # mains noise for the LMS noise reference


def load_ecg(filename):
    data = np.loadtxt(os.path.join(HERE, filename))
    fs = round(1 / (data[1, 0] - data[0, 0]))  # sampling rate from the time column (250 Hz)
    return fs, data[:, ECG_COLUMN] * 1000  #make mV


def save(fig, filename):
    fig.savefig(os.path.join(HERE, filename))


def spectrum(x, fs): # frequency axis
    N = len(x)
    X = 2 * np.abs(np.fft.fft(x)[:N // 2]) / N
    X[0] /= 2  # DC has no negative frequency twin
    return np.arange(N // 2) * fs / N, X


def db(X):
    return 20 * np.log10(X + 1e-12)


def plot_ecg(filename):
    fs, x = load_ecg(filename)
    t = np.arange(len(x)) / fs
    f, X = spectrum(x, fs)
    X = X * 1000  # makes uV

    # one heartbeat: R peak between 5 and 6 s, then 0.35 s before to 0.55 s after it
    r = 5 * fs + np.argmax(x[5 * fs:6 * fs])
    beat = slice(r - int(0.35 * fs), r + int(0.55 * fs))

    fig, ((ax1, ax2), (ax3, ax4)) = plt.subplots(2, 2, figsize=(11, 7))
    fig.suptitle(f"{filename} (raw, fs = {fs} Hz)")

    ax1.plot(t, x, lw=0.5)
    ax1.set_title("Whole recording")
    ax2.plot(t[beat], x[beat])
    ax2.set_title("One heartbeat")
```

```
    for ax in (ax1, ax2):
        ax.set_xlabel("Time (s)")
        ax.set_ylabel("ECG (mV)")

    # full spectrum scaled to the biggest peak above the wander band
    ax3.plot(f, X, lw=0.5)
    ax3.set_xlim(0, fs / 2)
    ax3.set_ylim(0, 1.2 * X[f > BASELINE[1]].max())
    ax3.axvspan(0.5, 40, color="g", alpha=0.15, label="ECG")
    ax3.axvspan(*BASELINE, color="r", alpha=0.3, label="Noise to remove")
    ax3.axvspan(*MAINS, color="r", alpha=0.3)
    ax3.set_title("Spectrum")


    # zoom on the low shows wander stoping and the heartbeat starting
    ax4.plot(f, X)
    ax4.set_xlim(0, 5)
    ax4.set_ylim(0, 1.3 * X[(f > BASELINE[1]) & (f < 5)].max())
    ax4.axvspan(*BASELINE, color="r", alpha=0.3, label="Noise: DC + baseline wander")
    #heart rate wanders (58-75 bpm), so its peak is smeared over this range rather than one line
    ax4.axvspan(40 / 60, 120 / 60, color="g", alpha=0.15, label="Heart rate, 40-120 bpm")
    ax4.set_title("Spectrum 0-5 Hz")
    ax4.legend(loc="upper right")
    for ax in (ax3, ax4):
        ax.set_xlabel("Frequency (Hz)")
        ax.set_ylabel("Amplitude (µV)")

    for ax in (ax1, ax2, ax3, ax4):
        ax.grid(alpha=0.3)
    fig.tight_layout()
    save(fig, filename.replace(".tsv", "_plot.pdf"))


if __name__ == "__main__":
    for filename in (ZABI, MINE):
        plot_ecg(filename)
    plt.show()
```

fir_design.py

```
import numpy as np


def design_fir(fs, stop_bands): #list contaning freqs bins to remove

    df = min(hi - lo for lo, hi in stop_bands) # minimum width of the stop bands
    # odd M with a symmetric h is a type 1 linear phase filter: the only type with no forced
    # zero at Nyquist, which the highpass needs, and the delay (M-1)/2 is a whole number of
    samples
    M = int(np.ceil(2 * fs / df)) | 1

    k = np.arange(M)
    f = np.minimum(k, M - k) * fs / M  # frequency of each bin, the top half mirrors the bottom
    H = np.ones(M)
    for lo, hi in stop_bands:
        H[(f >= lo) & (f <= hi)] = 0

    h = np.real(np.fft.ifft(H))
    h = np.roll(h, M // 2)  # swap the halves: negative time goes before positive time, so it is
    causal
    return h * np.hamming(M)  # smooth the truncation to cut the ripple
```

fir_filter.py

```python
import numpy as np


class FIRfilter:
    def __init__(self, _coefficients):
        self.h = np.array(_coefficients, dtype=float)
        self.M = len(self.h)
        self.buf = np.zeros(self.M)  # ring buffer for the delay line, samples never move once stored
        self.offset = 0  # where the next sample goes

    def store(self, v):
        # write v over the oldest sample and move the offset on, instead of shifting the buffer
        n = self.offset
        self.buf[n] = v
        self.offset = (n + 1) % self.M
        return n

    def halves(self, n):
        # the delay line newest first is buf[n], buf[n-1], ..., buf[0], then it wraps round to
        # buf[M-1], ..., buf[n+1]. h[0] lines up with the newest sample, so split h the same way
        return (self.h[:n + 1], self.buf[n::-1]), (self.h[n + 1:], self.buf[:n:-1])

    def dofilter(self, v):
        (h1, x1), (h2, x2) = self.halves(self.store(v))
        return np.dot(h1, x1) + np.dot(h2, x2)

    def doFilterAdaptive(self, signal, noise, learningRate):
        (h1, x1), (h2, x2) = self.halves(self.store(noise))
        e = signal - (np.dot(h1, x1) + np.dot(h2, x2))  # whatever the noise reference can't explain is the ECG
        # LMS: move each weight by its delay line sample times the error. h1, h2 are views, so this updates self.h
        h1 += learningRate * e * x1
        h2 += learningRate * e * x2
        return e
```

fir_realtime_sim.py

```python
"""Tasks 2 and 3: design the FIR filter and run it over our ECG one sample at a time.

Run with: python3 fir_realtime_sim.py
"""
import numpy as np
import matplotlib.pyplot as plt
from ecg_plot import MINE, BASELINE, MAINS, load_ecg, save, spectrum, db
from fir_design import design_fir
from fir_filter import FIRfilter


def plot_design(fs, h):
    M = len(h)
    print(f"Task 2: fs = {fs} Hz, {M} taps, delay {(M - 1) / 2 / fs:.2f} s")

    # impulse response and frequency response (zero padded FFT of h)
    Nf = 16 * M
    H = np.abs(np.fft.fft(h, Nf)[:Nf // 2])
    fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(8, 6))
    ax1.plot(np.arange(M) / fs, h)
    ax1.set_xlabel("Time (s)")
```

```
    ax1.set_ylabel("h(n)")
    ax1.set_title(f"Impulse response ({M} taps)")
    ax2.plot(np.arange(Nf // 2) * fs / Nf, db(H))
    ax2.set_xlim(0, fs / 2)
    ax2.set_ylim(-100, 10)
    ax2.set_xlabel("Frequency (Hz)")
    ax2.set_ylabel("Gain (dB)")
    ax2.set_title("Frequency response")
    fig.tight_layout()
    save(fig, "fir_design.pdf")


def plot_filtered(fs, x, y, M):
    delay = (M - 1) // 2
    t = np.arange(len(x)) / fs

    # whole recording before and after, skipping the first M samples while the delay line fills
    fig, (ax1, ax2, ax3) = plt.subplots(3, 1, figsize=(8, 9))
    ax1.plot(t, x)
    ax1.set_title("Raw ECG")
    ax2.plot(t[M:], y[M:])
    ax2.set_title("FIR filtered ECG")
    for ax in (ax1, ax2):
        ax.set_xlabel("Time (s)")
        ax.set_ylabel("ECG (mV)")
    for s, label in ((x - np.mean(x), "Raw"), (y[M:], "Filtered")):
        f, X = spectrum(s, fs)
        ax3.plot(f, db(X), label=label)
    ax3.set_xlim(0, fs / 2)
    ax3.set_xlabel("Frequency (Hz)")
    ax3.set_ylabel("Amplitude (dB re 1 mV)")
    ax3.set_title("Spectrum")
    ax3.legend()
    fig.tight_layout()
    save(fig, "fir_realtime_ecg.pdf")

    # one heartbeat, filtered output moved back by the filter delay so it lines up with the raw
    r = M + fs + np.argmax(np.abs(y[M + fs:M + 4 * fs]))  # an R peak in the filtered signal
    beat = np.arange(r - int(0.4 * fs), r + int(0.6 * fs))
    fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(8, 6), sharex=True)
    ax1.plot(t[beat - delay], x[beat - delay])
    ax1.set_title("Raw ECG, one heartbeat")
    ax2.plot(t[beat - delay], y[beat])
    ax2.set_title("FIR filtered ECG, one heartbeat (delay removed)")
    ax2.set_xlabel("Time (s)")
    for ax in (ax1, ax2):
        ax.set_ylabel("ECG (mV)")
    fig.tight_layout()
    save(fig, "fir_realtime_beat.pdf")


if __name__ == "__main__":
    fs, x = load_ecg(MINE)
    h = design_fir(fs, [BASELINE, MAINS])
    plot_design(fs, h)

    # feed the ECG in one sample at a time, like a realtime system would
    fir = FIRfilter(h)
    y = np.array([fir.dofilter(v) for v in x])
    plot_filtered(fs, x, y, len(h))
    plt.show()
```

lms_filter.py

```
"""Task 4: remove the mains with an adaptive LMS filter instead of the FIR bandstop, and compare.

Run with: python3 lms_filter.py
"""
import numpy as np
import matplotlib.pyplot as plt
from ecg_plot import MINE, BASELINE, MAINS, MAINS_HZ, load_ecg, save, spectrum, db
from fir_design import design_fir
from fir_filter import FIRfilter

fs, x = load_ecg(MINE)

# the fixed FIR from task 3, to compare against
bandstop = FIRfilter(design_fir(fs, [BASELINE, MAINS]))

# the LMS only gets a mains reference so baseline wander still needs the FIR highpass.
# it has the same number of taps as the bandstop (set by the baseline band) so the delays match
highpass = FIRfilter(design_fir(fs, [BASELINE]))
M = highpass.M

# noise reference: a sine at the measured mains frequency.
# one mains cycle of taps is enough to set the gain and phase of the mains sine
taps = fs // 50
lms = FIRfilter(np.zeros(taps))
# learning rate * taps * reference power (1/2) ~ 0.1: fast enough to follow the mains
# drifting, slow enough to leave the ECG alone (it goes unstable around 2)
learning_rate = 0.1 / (taps / 2)

# one sample at a time, like a realtime system would
y_fir = np.zeros(len(x))
y_lms = np.zeros(len(x))
for n, v in enumerate(x):
    y_fir[n] = bandstop.dofilter(v)
    ref = np.sin(2 * np.pi * MAINS_HZ * n / fs)
    y_lms[n] = lms.doFilterAdaptive(highpass.dofilter(v), ref, learning_rate)

t = np.arange(len(x)) / fs
r = M + fs + np.argmax(np.abs(y_fir[M + fs:M + 4 * fs]))  # an R peak to zoom in on
beat = slice(r - int(0.4 * fs), r + int(0.6 * fs))

fig, (ax1, ax2, ax3) = plt.subplots(3, 1, figsize=(8, 9))
ax1.plot(t[M:], y_fir[M:], label="FIR bandstop")
ax1.plot(t[M:], y_lms[M:], label="LMS", alpha=0.7)
ax1.set_title("Whole recording")
ax2.plot(t[beat], y_fir[beat], label="FIR bandstop")
ax2.plot(t[beat], y_lms[beat], label="LMS", alpha=0.7)
ax2.set_title("One heartbeat")
for ax in (ax1, ax2):
    ax.set_xlabel("Time (s)")
    ax.set_ylabel("ECG (mV)")
for s, label in ((y_fir[M:], "FIR bandstop"), (y_lms[M:], "LMS")):
    f, X = spectrum(s, fs)
    ax3.plot(f, db(X), label=label, alpha=0.7)
ax3.set_xlim(0, fs / 2)
ax3.set_xlabel("Frequency (Hz)")
ax3.set_ylabel("Amplitude (dB re 1 mV)")
ax3.set_title("Spectrum")
for ax in (ax1, ax2, ax3):
    ax.legend(loc="upper right")
fig.tight_layout()
save(fig, "lms_filter.pdf")
plt.show()
```

heartbeat.py

```python
"""Task 5: find the R peaks in our ECG with a matched filter and plot the momentary heart rate.

Run with: python3 heartbeat.py
"""
import numpy as np
import matplotlib.pyplot as plt
from ecg_plot import MINE, ZABI, BASELINE, MAINS, load_ecg, save
from fir_design import design_fir
from fir_filter import FIRfilter


fs, x = load_ecg(MINE)
fs_zabi, x_zabi = load_ecg(ZABI)
assert fs == fs_zabi, "template and ECG need the same sampling rate"
h = design_fir(fs, [BASELINE, MAINS])
M = len(h)

# setup, done once before the realtime loop like designing the coefficients:
# the template is one PQRST from Zabi's filtered ECG
fir = FIRfilter(h)
y_zabi = np.array([fir.dofilter(v) for v in x_zabi])
r = M + fs + np.argmax(np.abs(y_zabi[M + fs:M + 4 * fs]))
template = y_zabi[r - int(0.3 * fs):r + int(0.4 * fs)]
template = template - np.mean(template)

# the threshold is fixed before any of our ECG arrives. When the template meets itself the matched
# filter gives sum(template^2), squared below. Our beats are a bit smaller than Zabi's and peak at
# about 0.45 of that, so 0.15 leaves a safe margin
threshold = 0.15 * np.sum(template ** 2) ** 2

# realtime: every sample goes through the bandstop, then the matched filter (time reversed
    template),
# then gets squared, which makes the peaks stand out and doesn't care about the sign.
# A beat is an upward crossing of the threshold. Nothing here looks ahead in the recording
fir = FIRfilter(h)
matched = FIRfilter(template[::-1])
skip = M + len(template)  # both delay lines filling up
min_gap = 60 / 200 * fs  # heuristic: the heart can't beat faster than 200 bpm
m = np.zeros(len(x))
beats, rejected = [], []
for n, v in enumerate(x):
    m[n] = matched.dofilter(fir.dofilter(v)) ** 2
    if n > skip and m[n] > threshold >= m[n - 1]:
        # a crossing too soon after the last beat is noise. dropping the detection (not just the
        # rate) keeps the interval to the next real beat right
        if beats and n - beats[-1] < min_gap:
            rejected.append(n)
        else:
            beats.append(n)
beats = np.array(beats)
detections = np.sort(beats.tolist() + rejected)
print(f"Task 5: {len(detections)} detections, {len(rejected)} rejected")


def heart_rate(b):
    return b[1:] / fs, 60 / (np.diff(b) / fs)  # bpm at the time of each beat


t = np.arange(len(x)) / fs
fig, (ax1, ax2, ax3) = plt.subplots(3, 1, figsize=(8, 9))
ax1.plot(np.arange(len(template)) / fs, template)
ax1.set_xlabel("Time (s)")
```

```
ax1.set_ylabel("ECG (mV)")
ax1.set_title("Template (Zabi's heartbeat)")
show = slice(skip, skip + 10 * fs)
ax2.plot(t[show], m[show])
ax2.axhline(threshold, color="r", linestyle="--", label="Threshold")
b = beats[beats < skip + 10 * fs]
ax2.plot(t[b], m[b], "rx", label="Detected beat")
ax2.set_xlabel("Time (s)")
ax2.set_ylabel("Matched filter output$^2$")
ax2.set_title("R-peak detection")
ax2.legend(loc="upper right")
ax3.plot(*heart_rate(detections), "x", color="grey", label="Before heuristic")
ax3.plot(*heart_rate(beats), label="After heuristic")
ax3.set_xlabel("Time (s)")
ax3.set_ylabel("Heart rate (bpm)")
ax3.set_title("Momentary heart rate")
ax3.legend(loc="upper right")
fig.tight_layout()
save(fig, "heartbeat.pdf")
plt.show()
```